Lecture 28.11: Post-Pruning Implementation (Cost-Complexity ccp_alpha)

📌 Topic Overview

In this lecture, we study **Post-Pruning** using **Minimal Cost-Complexity Pruning** (`ccp_alpha`). We let the tree grow completely, compute the pruning path with `cost_complexity_pruning_path()` , and find the optimal alpha parameter that maximizes generalization.

📐 Mathematical Formulation: Cost-Complexity Cost Function

Cost-complexity pruning defines a cost function that balances classification error with tree size:

$$R_\alpha(T) = R(T) + \alpha|T|$$

Where:

- $R(T)$ = Total training misclassification error of tree T.
- $|T|$ = Number of terminal leaf nodes in tree T (a measure of tree complexity).
- $\alpha \geq 0$ = Complexity penalty parameter (`ccp_alpha`).
- When $\alpha = 0$: Standard unpruned tree (pure leaves, high complexity).
- As $\alpha \to \infty$: Penalty dominates; the tree gets pruned all the way back to a single root node.

🛠️ Step-by-Step Implementation from Lecture

1. Extract the Pruning Path

Scikit-Learn provides `cost_complexity_pruning_path()` , which returns the exact candidate alpha values where the tree structure changes:

```
path = model.cost_complexity_pruning_path(X_train, y_train)
ccp_alphas, impurities = path.ccp_alphas, path.impurities
```

2. Train a Model for Each Candidate Alpha

In the video, the instructor loops through `ccp_alphas`:

```
trees = []
for alpha in ccp_alphas:
model = DecisionTreeClassifier(random_state=42, ccp_alpha=alpha)
model.fit(X_train, y_train)
acc = model.score(X_test, y_test)
trees.append((model, acc, alpha))
```

3. Find the Best Alpha

We select the alpha that gives the maximum test accuracy:

```
best_tree, best_acc, best_alpha = max(trees, key=lambda x: x[1])
print(f"Optimal ccp_alpha: {best_alpha:.5f}")
print(f"Best Test Accuracy: {best_acc:.4f}")
```

💻 Python Implementation

```
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier

# Fit initial tree to get pruning path
clf_initial = DecisionTreeClassifier(random_state=42)
path = clf_initial.cost_complexity_pruning_path(X_train, y_train)
ccp_alphas = path.ccp_alphas

train_scores = []
test_scores = []

for alpha in ccp_alphas:
clf = DecisionTreeClassifier(random_state=42, ccp_alpha=alpha)
clf.fit(X_train, y_train)
train_scores.append(clf.score(X_train, y_train))
test_scores.append(clf.score(X_test, y_test))

# Best alpha selection
best_idx = np.argmax(test_scores)
print(f"Best Alpha: {ccp_alphas[best_idx]:.5f}")
print(f"Best Test Accuracy: {test_scores[best_idx]:.4f}")
```

🔑 Key Takeaways

1. Post-pruning lets the tree grow fully, then trims weakest leaves bottom-up.
2. `cost_complexity_pruning_path()` provides exact candidate `ccp_alpha` thresholds.
3. Increasing `ccp_alpha` simplifies the tree by penalizing the number of leaf nodes.